

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**СИСТЕМА ЗА ОБРАБОТКА НА ОБЛАЦИ ОТ
ТОЧКИ И КАРТИ НА ДЪЛБОЧИНАТА С
УСКОРЕНО ТЪРСЕНЕ ВЪРХУ
ПРОСТРАНСТВЕН ИНДЕКС**

КУРСОВ ПРОЕКТ

по дисциплина „Компютърно зрение“

Разработил:

Алекс Цветанов, фак. № **[TODO: фак. номер]**

Специалност: Компютърно и софтуерно инженерство, ОКС „Магистър“

Преподавател:

Проф. д-р инж. Милена Лазарова

София, **[TODO: година на предаване]**

СЪДЪРЖАНИЕ

УВОД	1
I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД .	3
II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА	5
2.1. Пространствен индекс	5
2.2. Стратегия за векторизация	5
2.3. Организация на конвейера и роля на външните библиотеки	6
III. ПРОЕКТИРАНЕ НА СИСТЕМАТА	7
IV. ПРОГРАМНА РЕАЛИЗАЦИЯ	8
V. МЕТОДИКА НА ЕКСПЕРИМЕНТАЛНАТА ПРОВЕРКА	9
VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ	10
6.1. Съвпадение с еталона	10
6.2. Цена на заявката за съседи	10
6.3. Ефект на векторизацията	10
6.4. Цена на конвейера	10
6.5. Анализ	10
ЗАКЛЮЧЕНИЕ	11
ЛИТЕРАТУРА	12
ПРИЛОЖЕНИЯ	14

УВОД

Сензорите за дълбочина и стереоскопичните камери извеждат данни, които не приличат на обикновено растерно изображение. Резултатът е *облак от точки*: неподредено множество от координати в тримерното пространство, чиито съседства не се четат от индекса на пиксела, а трябва да бъдат намерени. Почти всяка стъпка от обработката, от премахването на шума през оценяването на нормалите и извличането на признаци до съвместяването на два припокриващи се записа, се свежда до един и същ въпрос: кои точки лежат близо до дадена точка. Затова цената на целия конвейер се определя не толкова от аритметиката на отделния алгоритъм, колкото от това колко бързо се отговаря на този въпрос.

Целта на курсовия проект е да се проектира и реализира конвейер за обработка на облаци от точки и карти на дълбочината, в който търсенето на съседи е изнесено в отделен, ускорен компонент върху пространствен индекс, и всяка стъпка от конвейера е измерима самостоятелно. Реализацията е на C++20, а ускорението използва явна векторизация на ниво инструкции (SIMD) върху подредени по памет данни.

За постигането на целта са формулирани следните **задачи**:

1. преглед на апаратната и информационната структура на система за компютърно зрение и на класическите методи за предварителна обработка, филтриране, откриване на контури, извличане на признаци и сегментация;
2. анализ на възможните решения за пространствен индекс, за стратегия на векторизация и за организация на конвейера, и обосновка на направения избор;
3. проектиране на конвейер със самостоятелно измерими стъпки и с ясна граница между преносимата логика и векторизираната част;
4. програмна реализация на стъпките: филтриране, оценяване на нормали, извличане на признаци, сегментация на еднородни области, оценяване на дълбочина по стереодвойка и съвместяване на облаци от семейството на итеративната най-близка точка;
5. изграждане на методика за измерване, която сравнява всяка стъпка по точност и по време спрямо еталонна реализация;
6. провеждане на измерванията и анализ на получените резултати.

Обхватът на проекта е геометричната и фотометричната обработка: всичко от

входните данни до съвместените облаци и сегментираните области. Извън обхвата съзнателно остават разпознаването по обучени модели и невронните мрежи. Причината не е липса на интерес, а несъвместима методика на оценяване: качеството на обучен разпознавател зависи преди всичко от набора от данни и от процедурата на обучение, а предмет на измерване тук е цената на геометричната обработка. Двете не се измерват с един и същ експеримент, а обем от един курсов проект не стига и за двете. Тези теми са разгледани в литературния преглед и мястото им в конвейера е посочено, но не са реализирани.

Структура на изложението. Раздел I разглежда предметната област и методите, върху които стъпва работата. Раздел II излага разгледаните алтернативи и обосновава избора между тях. Раздел III представя проектирането на конвейера, а Раздел IV неговата програмна реализация. Раздел V описва методиката на измерване, а Раздел VI получените резултати. Заключението обобщава постигнатото и очертава посоките за продължение.

I. АНАЛИЗ НА ПРЕДМЕТНАТА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД

Системата за компютърно зрение се описва обичайно като верига от източник на данни, слой за предварителна обработка, слой за извличане на признаци и слой за вземане на решение. Апаратната част определя вида и дефектите на входа: пасивната стереодвойка дава плътна карта на дълбочината само там, където сцената има текстура, а активните сензори със структурирана светлина или с време на прелитане дават плътност, независима от текстурата, но внасят шум по краищата на обектите и загуба на данни при отразяващи повърхности. Информационната структура надолу по веригата е следствие от този избор: подредено изображение на дълбочината запазва съседството по индекс, докато облакът от точки го губи и налага пространствен индекс.

Предварителната обработка и филтрирането целят потискане на шума без разрушаване на геометрията. Линейното изглаждане размива именно ръбовете, които следващите стъпки търсят, затова се предпочитат методи, съобразени с ръба: двустранното филтриране претегля съседите едновременно по разстояние и по разлика в стойността [1], а при облаци от точки същата идея се прилага върху разстоянието до допирателната равнина. Откриването на контури върху изображението на интензитета следва класическата постановка на Кани с трите ѝ критерия за добро откриване, добра локализация и единичен отговор [2]; върху карта на дълбочината същият апарат откроява геометричните прекъсвания, но изисква отделно третиране на липсващите стойности.

Извличането на признаци и сегментацията са мястото, където подредените и неподредените данни се разминават. За облаци от точки утвърдено средство са хистограмите на бързите точкови признаци, които описват околността на точка чрез разпределение на ъглите между нормалите на съседите ѝ и са проектирани така, че цената им да расте линейно с броя на точките [3]. Сегментацията на еднородни области се води или от геометрията, чрез съгласуваност на нормалите и кривината, или от подгонване на параметричен модел към подмножество от точки. Вторият подход почти винаги стъпва върху устойчиво оценяване със случайни извадки [4], което остава работещо при голям дял груби грешки. Текстурната сегментация на изображението ползва отделно семейство описания, класическото сред които са статистиките от матрицата на съвместната поява на нива на сивото [5].

Анализът на тримерната сцена и стереодвойката са двете страни на един и същ въпрос за възстановяване на дълбочината. Полуглобалното съгласуване свежда двумерната задача за съответствие до сумиране на едномерни динамични програми по няколко направления и е приетият компромис между качеството на глобалните и цената на локалните методи [6]. Когато сцената се сменя от няколко положения, получените облаци трябва да бъдат приведени в обща координатна система. Каноничното решение е итеративната най-близка точка [7], чиито варианти се различават по избора на съответствия, по метриката и по стратегията на отхвърляне; систематичното им сравнение показва, че тези решения влияят на скоростта на сходимост повече от самата формулировка [8]. Всяка итерация изисква търсене на най-близък съсед, което връща изложението към структурата на данните: k-d дървото [9] остава основното средство, а приблизителните варианти купуват скорост срещу контролирана загуба на точност [10].

Разпознаването по модел и подходите с невронни мрежи заемат последното ниво на веригата и са извън реализацията по причината, изложена в увода. Тук те се отбелязват като потребител на изхода от настоящия конвейер: сегментираните области и описанията на околностите са точно входът, който един разпознавател очаква, независимо дали е геометричен или обучен. Утвърдените реализации с отворен код, спрямо които се сравнява настоящата работа, са библиотеката за облаци от точки [11] и библиотеката за компютърно зрение [12].

II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА

2.1. Пространствен индекс

Три решения са приложими за търсене на съседи в облак от точки. Вокселната решетка разделя пространството на кубчета с постоянен размер и намира съседите чрез обхождане на съседните кубчета. Тя е тривиална за построяване и изключително бърза при равномерна плътност, но при неравномерна плътност или силно издължена сцена или изразходва памет за празни кубчета, или събира прекалено много точки в едно. K-d дървото [9] се приспособява към плътността, защото разделя по медиана, и дава предсказуемо поведение при заявка за k най-близки съседи, но обхождането му е с нередовен достъп до паметта. Октодървото стои между двете и се предпочита, когато е нужна работа на няколко разделителни способности едновременно.

Изборът е k-d дърво като основен индекс и вокселна решетка като второ, независимо изпълнение на същия интерфейс. Причината за две реализации е методическа, а не инженерна: измерването на едната срещу другата върху едни и същи данни превръща избора от твърдение в резултат. Приблизителните схеми [10] са разгледани и оставени извън обхвата, защото внасят втори източник на грешка, който би се смесил с грешката на измервания алгоритъм.

2.2. Стратегия за векторизация

Ускоряването на заявките има три възможни пътя. Разчитане на автоматичната векторизация на компилатора е най-евтино откъм усилие, но резултатът зависи от компилатора и от неговата версия и не е част от договора на езика. Вътрешните функции на конкретния набор инструкции дават пълен контрол, но обвързват изходния текст с една архитектура и налагат отделна реализация за всяка. Библиотека за преносима векторизация запазва единен изходен текст и оставя избора на набор инструкции на времето за компилация.

Избран е третият път. Той е съвместим и с преминаване към стандартната поддръжка на кратки вектори, когато тя стане достъпна в използваните компилатори. Условието за това да работи е разположението на данните: заявките се векторизират

смислено само ако координатите са подредени по компонента, а не по точка. Това превръща оформлението на паметта в проектно решение, а не в подробност от реализацията, и е разгледано в Раздел III.

2.3. Организация на конвейера и роля на външните библиотеки

Конвейерът може да бъде написан като монолитна функция, като верига от обекти с общ интерфейс, или като съвкупност от независими преобразувания, свързвани при конфигурирането. Първото е най-кратко, но прави отделната стъпка неизмерима. Избрано е второто: всяка стъпка е обект с еднакъв договор, който приема облак и връща облак заедно с показатели за изпълнението си. Така времето на стъпката е наблюдаемо без промяна в кода, а редът на стъпките се задава отвън.

Външните библиотеки участват само в две роли: вход и изход на данните, и еталон за сравнение [11, 12]. Алгоритмите, които са предмет на изследването, се реализират в проекта. Обратното решение, тоест извикване на готовите реализации, би направило измерването безсмислено, защото щеше да сравнява библиотека със самата себе си. **[TODO: потвърди с преподавателя, че използването на еталонна библиотека само за сравнение е допустимо в рамките на заданието]**

III. ПРОЕКТИРАНЕ НА СИСТЕМАТА

Системата е разделена на четири слоя. Най-долният съдържа представянето на данните: облак от точки, карта на дълбочината и помощните описания на околност. Над него стои пространственият индекс с единен интерфейс за заявка по k най-близки съседи и за заявка по радиус. Третият слой съдържа стъпките на обработката, всяка от които зависи само от долните два. Най-горният слой е конвейерът, който подрежда стъпките, подава им конфигурация и събира показателите от изпълнението.

Разположението на данните е проектно решение, не подробности. Облакът се съхранява като отделни непрекъснати масиви за всяка координата, а не като масив от тройки. Така една векторна инструкция обработва няколко точки наведнъж без разместване, а достъпът до една координата на много точки, който е типичният достъп при заявка, чете последователна памет. Цената е по-неудобен достъп до отделна точка и необходимост от изрично поддържане на съгласуваност между масивите, затова достъпът минава през тесен интерфейс, който не позволява на потребителя да види вътрешното представяне.

Интерфейсът на индекса е умишлено тесен: построяване от облак, заявка за k най-близки съседи и заявка по радиус, плюс отчитане на изразходваната памет. Всичко останало, включително оценяването на нормали и извличането на признаци, е потребител на този интерфейс, а не негова част. Това дава и заменяемост на реализациите, което методиката в Раздел V използва пряко.

Всяка стъпка от обработката приема облак и конфигурация и връща нов облак заедно със запис на изпълнението си. Стъпките не променят входа си на място. Решението е за сметка на паметта, но прави възможно повтарянето на една стъпка с различни параметри без повторно изпълнение на предходните и премахва скрита зависимост между стъпките, която би направила измерването на отделна стъпка невъзможно.

[TODO: диаграма на слоевете и на потока от данни през конвейера (Фиг.), с легенда]

[TODO: описание на формата на конфигурационния файл на конвейера, след като бъде фиксиран]

IV. ПРОГРАМНА РЕАЛИЗАЦИЯ

Изходният текст е на C++20 и се изгражда с CMake. Разделен е на заглавни файлове с публичните типове и на единици за превод с реализацията, така че смяна на реализацията на индекса да не води до преизграждане на целия проект. Векторизираната част е събрана в отделни файлове, за да може да бъде компилирана с различни настройки от останалия код и да бъде изключена изцяло при сравнение със скаларната реализация.

Ядрото на реализацията е заявката за най-близки съседи. При k -d дървото обхождането поддържа купчина с текущите най-добри кандидати и отсича поддървета, чието разстояние по разделящата равнина вече надхвърля най-лошото прието разстояние. Векторизацията се прилага на нивото на листата: точките в един лист се проверяват на групи, вместо поотделно, което е възможно именно заради разположението по компонента, описано в Раздел III. Размерът на листа става настройка, защото балансира дълбочината на обхождане срещу дължината на векторизираната част.

Стъпките на обработката ползват индекса през общия му интерфейс. Филтрирането по статистика на разстоянията до съседите, оценяването на нормали чрез анализ на главните компоненти на околността, извличането на описания на околност [3], сегментацията чрез нарастване на области по съгласуваност на нормалите и подгонването на равнина с устойчиво оценяване [4] са реализирани като отделни стъпки с еднакъв договор. Съвместяването следва схемата на итеративната най-близка точка [7] с метрика до точката и с отхвърляне на съответствията над зададен праг [8].

[TODO: йерархия на типовете (Фиг.), с легенда]

[TODO: листинг на заявката за k най-близки съседи, вмъкнат с `\lstinputlisting` от `src/`, след като реализацията бъде завършена]

[TODO: описание на модулите за вход и изход и на поддържаните формати на файловете]

V. МЕТОДИКА НА ЕКСПЕРИМЕНТАЛНАТА ПРОВЕРКА

Измерването отговаря на два отделни въпроса и затова е разделено на две части. Първата е за коректност: дава ли всяка стъпка същия резултат като еталонната реализация на същия алгоритъм. Втората е за цена: колко време и колко памет отнема стъпката. Двете не се смесват, защото по-бърза стъпка с различен резултат не е по-бърза, а друга стъпка.

Еталон за сравнение са реализациите на същите стъпки в утвърдените библиотеки с отворен код [11, 12]. За заявката за най-близки съседни еталонът е изчерпателно търсене, чийто резултат е точен по определение и служи за проверка на индекса. Съвпадението се проверява по множеството от върнати индекси, а при равни разстояния се допуска различен ред, защото редът не е част от договора.

Измерваните величини са: време за построяване на индекса, време за заявка при няколко стойности на k и на радиуса, изразходвана памет от индекса, и време на всяка стъпка от конвейера поотделно. Всяко измерване се повтаря многократно и се отчита медианата заедно с разсейването, защото средното е чувствително към единични отклонения от планировчика на операционната система.

[TODO: набор от данни: източник, брой облаци, брой точки, начин на снемане. Използват се само публично достъпни набори и не се съобщава характеристика, която не е проверена в оригиналния източник]

[TODO: конфигурация на машината: процесор, набор инструкции, размер на кеша, памет, компилатор и версия, флагове за оптимизация]

[TODO: протокол на изпълнение: брой повторения, загряващи изпълнения, състояние на мащабирането на честотата]

Таблиците с резултати имат предварително фиксирана форма, за да не бъде изборът им подчинен на това, което измерването е показало. Таблица 1 съпоставя двете реализации на индекса по време за построяване, време за заявка и памет. Таблица 2 дава времето на всяка стъпка от конвейера при скаларна и при векторизирана реализация. Таблица 3 съпоставя резултата на всяка стъпка с еталона по избраната за нея мярка за съвпадение. Всички клетки остават празни до провеждането на измерването.

VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ

Разделът представя резултатите от измерванията по методиката от Раздел V. Редът следва реда на въпросите: първо съвпадението с еталона, защото без него измерването на времето няма смисъл, после цената на заявката, после цената на конвейера като цяло.

6.1. Съвпадение с еталона

[TODO: Таблица: стъпка, мярка за съвпадение, резултат, заключение. Попълва се от изхода на проверката, не на ръка. В текста задължително има коментар и препратка към таблицата]

6.2. Цена на заявката за съсед

[TODO: Таблица: реализация на индекса, брой точки, време за построяване, време за заявка при зададените стойности на k , изразходвана памет]

[TODO: Фигура: време за заявка спрямо броя на точките, за двете реализации на индекса, с легенда и препратка в текста]

6.3. Ефект на векторизацията

[TODO: Таблица: стъпка, скаларна реализация, векторизирана реализация, отношение. Отношението се изчислява от измерените стойности, не се заявява предварително]

6.4. Цена на конвейера

[TODO: Таблица: стъпка от конвейера, дял от общото време. Придружава се с коментар кой етап доминира и дали това съответства на очакването от Раздел III]

6.5. Анализ

[TODO: Анализ на резултатите. Пише се СЛЕД като таблиците са попълнени. Всяко твърдение тук трябва да сочи към клетка от таблица. Отклоненията от очакването се отчитат като отклонения, а не се обясняват с допълнителни предположения]

ЗАКЛЮЧЕНИЕ

Проектът разглежда конвейер за обработка на облаци от точки и карти на дълбочината, в който търсенето на съседи е обособено като отделен компонент върху пространствен индекс и е ускорено чрез явна векторизация. Обособяването има две последици. Първата е инженерна: всяка стъпка от конвейера ползва един и същ тесен интерфейс, така че реализацията на индекса се сменя без промени в стъпките. Втората е методическа: цената на всяка стъпка се измерва самостоятелно, а двете реализации на индекса се сравняват върху едни и същи данни.

Основното ограничение на решението е, че печалбата от векторизацията зависи от разположението на данните по компонента, а то прави достъпа до отделна точка по-скъп. Изборът е изгоден за конвейер, в който преобладават заявките по околност, и неизгоден за натоварване, в което се четат единични точки в произволен ред. Второ ограничение е съзнателното изключване на разпознаването по обучени модели, което оставя последното ниво на веригата извън обхвата и е изложено в увода.

[TODO: обобщение на измерените резултати. Пише се СЛЕД попълването на Раздел \ref{sec:results} и не съдържа твърдение, което не се вижда в таблица оттам]

Посоките за продължение са три. Първата е приблизително търсене на съседи с контролирана загуба на точност [10], което е разгледано и оставено извън обхвата именно защото внася втори източник на грешка. Втората е паралелно изпълнение на стъпките върху няколко ядра, за което текущият договор на стъпката вече е подходящ, понеже стъпките не променят входа си на място. Третата е добавяне на разпознавател над изхода на конвейера, което би затворило веригата от сензор до решение.

ЛІТЕРАТУРА

- [1] C. Tomasi and R. Manduchi, “Bilateral filtering for gray and color images,” in *Sixth International Conference on Computer Vision (ICCV)*, pp. 839–846, 1998.
- [2] J. Canny, “A computational approach to edge detection,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 8, no. 6, pp. 679–698, 1986.
- [3] R. B. Rusu, N. Blodow, and M. Beetz, “Fast point feature histograms (FPFH) for 3D registration,” in *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 3212–3217, 2009.
- [4] M. A. Fischler and R. C. Bolles, “Random sample consensus: A paradigm for model fitting with applications to image analysis and automated cartography,” *Communications of the ACM*, vol. 24, no. 6, pp. 381–395, 1981.
- [5] R. M. Haralick, K. Shanmugam, and I. Dinstein, “Textural features for image classification,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-3, no. 6, pp. 610–621, 1973.
- [6] H. Hirschmüller, “Stereo processing by semiglobal matching and mutual information,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 30, no. 2, pp. 328–341, 2008.
- [7] P. J. Besl and N. D. McKay, “A method for registration of 3-D shapes,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 14, no. 2, pp. 239–256, 1992.
- [8] S. Rusinkiewicz and M. Levoy, “Efficient variants of the ICP algorithm,” in *Proceedings of the Third International Conference on 3-D Digital Imaging and Modeling*, pp. 145–152, 2001.
- [9] J. L. Bentley, “Multidimensional binary search trees used for associative searching,” *Communications of the ACM*, vol. 18, no. 9, pp. 509–517, 1975.
- [10] M. Muja and D. G. Lowe, “Scalable nearest neighbor algorithms for high dimensional data,” *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 36, no. 11, pp. 2227–2240, 2014.

- [11] R. B. Rusu and S. Cousins, “3D is here: Point Cloud Library (PCL),” in *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1–4, 2011.
- [12] G. Bradski, “The OpenCV library,” *Dr. Dobb’s Journal of Software Tools*, vol. 25, no. 11, pp. 120–125, 2000.

ПРИЛОЖЕНИЯ

Приложение А. Структура на изходния текст

[TODO: дърво на директориите на хранилището с кратко описание на всяка, генерирано след като реализацията придобие окончателния си вид]

Приложение Б. Формати на структурите от данни

[TODO: описание на представянето на облака от точки и на индекса в паметта, с размери на полетата]

Приложение В. Избрани части от изходния текст

[TODO: листинги, вмъкнати с \lstinputlisting от src/, за да не се разминават с кода. Не се преписват на ръка]

Приложение Г. Как се възпроизвеждат измерванията

[TODO: точните команди за изграждане и за пускане на измерванията, заедно с версиите на инструментите, с които са получени резултатите от Раздел \ref{sec:results}]